

Search Typeahead System

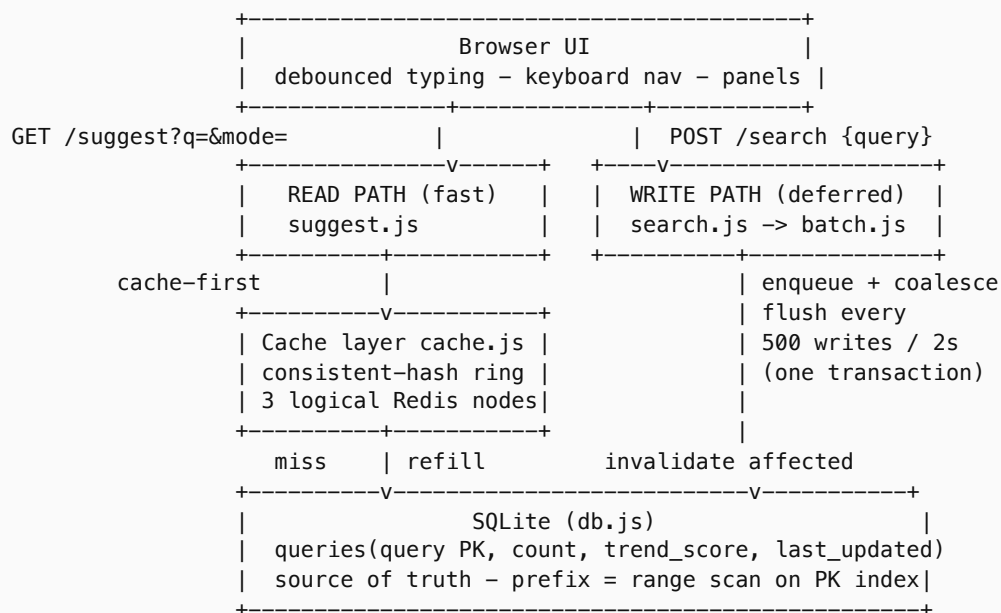
Project Report — distributed cache · consistent hashing · recency-aware trending · batched writes

Repository: github.com/varsharaodevaraj/search-typeahead · Stack: Node.js + Express, SQLite, Redis, vanilla JS

1. Architecture

A search-typeahead backend and UI. As the user types, the system suggests popular queries; on submit it records the query and updates its popularity. The design focus is the data system: how query counts are stored, how suggestions are served with low latency, how the cache is distributed with consistent hashing, and how write pressure is reduced with batching.

Two request paths share one durable store. The **read path** is cache-first; the **write path** is deferred and batched.



Components

- **db.js (SQLite source of truth):** one table keyed by query text. Prefix matching is a range scan — every string starting with prefix `p` sorts inside `[p, p+' ']`, so the scan rides the primary-key index. No in-memory trie to rebuild or lose on restart, and read/write counts are measurable.
- **ring.js (consistent-hash ring):** each node is scattered as 150 virtual points on a 2^{32} ring; a key is owned by the first virtual node clockwise from `md5(key)`. Even load, and adding/removing a node remaps only $\sim 1/N$ of keys.
- **cache.js (distributed cache):** N logical Redis nodes behind the ring. Key `suggest:<mode>:<prefix>`; the prefix is hashed so both modes co-locate and invalidate together. Every call is wrapped — a down node is a miss and the read falls back to SQLite (graceful degradation).
- **batch.js (write path):** buffers searches, coalesces repeats, flushes in one transaction every 500 writes or 2s, snapshot-then-write so mid-flush searches are not lost.

- **trending.js (recency-aware ranking):** lazily decayed `trend_score` blended with popularity.
- **server.js:** Express routes plus graceful-shutdown flush.

2. Dataset — source & loading

Source: Peter Norvig's public word-frequency lists, derived from the Google Web Trillion Word Corpus — `count_1w.txt` (~333k single words) and `count_2w.txt` (~286k two-word phrases).

Loading: `npm run load-data` downloads the lists, merges the unigrams with the top-100k bigrams, scales the raw counts down (so a live `+1` search is meaningful), and bulk-inserts into SQLite. Result: **424,188 rows** (well above the 100k minimum). If the network is unavailable it falls back to a synthetic Zipf-distributed dataset, so the loader never hard-fails.

query	count
iphone	100000
iphone 15	85000
iphone charger	60000

Run order: `npm install` → `brew services start redis` → `npm run load-data` → `npm start` (<http://localhost:3000>).

3. API documentation

Method & path	Purpose	Behavior
<code>GET /suggest?q=<prefix>&mode=</code>	fetch suggestions	Up to 10 prefix matches. <code>mode=basic</code> → by all-time count; <code>mode=trending</code> (default) → recency-aware blend. Returns <code>{query, mode, cache:hit miss, node, suggestions[]}</code> .
<code>POST /search {query}</code>	submit a search	Returns <code>{message:"Searched", query, recorded}</code> and enqueues a batched count update.
<code>GET /trending?limit=N</code>	global hot feed	Currently-trending queries (all-time top before any activity).
<code>GET /cache/debug?prefix=<p></code>	debug routing	Which cache node owns the prefix and whether it is a hit/miss.
<code>GET /stats</code>	live metrics	DB rows + read/write counts, cache hit rate per node, suggest p50/p95/p99 latency, batch write-reduction.

4. Design choices & trade-offs

- **SQLite range scan, not a trie.** Correct, durable across restarts, and measurable. A trie is faster in memory but lost on restart and harder to instrument.
- **Cache-first reads.** Only a miss touches SQLite; the result is written back. This keeps p95 latency sub-millisecond.

- **Consistent hashing on the prefix.** 150 vnodes/node give even load; adding/removing a node remaps only $\sim 1/N$ of keys (vs. nearly all for `hash % N`). Hashing the prefix co-locates a prefix's two modes so invalidation is single-node.
- **Batched writes.** Repeats are coalesced and applied in one transaction per window. *Trade-off:* a clean shutdown flushes first (zero loss); a hard crash loses only the un-flushed buffer (≤ 500 searches, i.e. a few popularity counts) — acceptable for a popularity counter, and the price of not writing synchronously.
- **Recency-aware trending.** `score =  $\alpha \cdot \text{normCount} + (1-\alpha) \cdot \text{normRecency}$` ($\alpha = 0.5$), with lazy exponential decay (1-hour half-life). A brief spike decays below a floor and stops ranking, so nothing is over-ranked forever. The cache is invalidated on every flush so rankings stay fresh.
- **Graceful degradation.** If Redis is down, reads fall back to SQLite — the system slows, it never breaks.

5. Performance report

Measured by `npm run bench` against a local server (3 logical Redis nodes, 424k-row dataset): 2,000 `/suggest` reads (repeated prefixes, so the cache warms) and 1,000 `/search` writes, then the server's own `/stats`.

0.101 ms	98.95%	24.4x
suggest p95 latency	cache hit rate	write reduction

Suggest latency (read path)

percentile	latency
p50	0.075 ms
p95	0.101 ms
p99	0.457 ms

Over 2,002 requests. The cache-first path keeps typical reads well under 1 ms; the p99 tail is cache misses that fall through to a SQLite range scan.

Cache hit rate

metric	value
hits / misses	1,981 / 21
hit rate	98.95%
routing across nodes (hits)	495 / 792 / 694
virtual nodes per physical node	150 / 150 / 150

$\sim 99\%$ of reads were served from Redis without touching SQLite. Misses are cold-cache first lookups and reads right after a write invalidated that prefix.

Write reduction (batching)

metric	value
searches submitted	1,001
SQLite row writes	41
transactions (flushes)	3
write reduction	24.4×
transaction reduction	333.7×

1,001 individual searches became 41 row writes across 3 transactions — repeats coalesced into single `+delta` upserts. Confirmed by `dbIO.writes = 41` on `/stats`. Note: a fresh cold-start session shows lower hit rate and higher p95 until the cache warms; the numbers above are the steady-state benchmark.